AI-Based Behavioral Anomaly Detection for Personal Devices

Comprehensive Project Report

1. Executive Summary

The AI-Based Behavioral Anomaly Detection system is a next-generation, privacy-first cybersecurity platform designed for personal devices. By combining ultra-low-overhead kernel telemetry with local Large Language Models (LLMs) via Ollama, the system detects, analyzes, and mitigates zero-day threats and sophisticated behavioral anomalies in real-time, entirely on-device.

2. The Problem Statement

Traditional antivirus solutions rely heavily on static signatures and heuristic engines that struggle against obfuscated malware, living-off-the-land (LotL) binaries, and zero-day exploits. Cloud-based EDR (Endpoint Detection and Response) solutions solve this via complex behavioral analysis but require transmitting sensitive personal compute telemetry to external servers. There is a critical need for an endpoint security solution that provides enterprise-grade behavioral analysis without compromising user privacy or consuming excessive local compute resources.

3. What is the Project?

This project is an autonomous, on-device security orchestrator. It acts as an intelligent, locally-hosted security analyst that monitors system behavior, filters out the noise, and uses advanced AI reasoning to evaluate suspicious activities. It is built specifically for power users, developers, and security engineers who require deep, kernel-level visibility and offensive-grade defensive capabilities tailored to their exact workflow.

4. How Does It Work? (Core Architecture)

Because an LLM cannot process thousands of raw system calls per second, the architecture operates in a tiered pipeline, balancing high-speed data ingestion with heavy AI reasoning:

- **Tier 1: Deep Kernel Telemetry (The Sensor):** Using eBPF (Extended Berkeley Packet Filter) written in C and managed by a Go daemon, the system hooks directly into kernel tracepoints (e.g., `sys_enter_execve`, `sys_enter_connect`). This captures raw process creations, file modifications, and network sockets with near-zero overhead.

- **Tier 2: Machine Learning Triage (The Pre-Processor):** Raw telemetry is streamed instantly via WebSockets to a Python/FastAPI backend. A fast, unsupervised ML model (Isolation Forest) trained on the user's localized baseline evaluates the events. Normal behavior is logged and ignored; statistical deviations trigger the next tier.
- **Tier 3: AI Reasoning Engine (The Analyst):** Suspicious process trees and network connections are batched and formatted into context-rich prompts. These are sent to a local LLM (like Llama 3 8B or Qwen 2.5) running via Ollama. The LLM performs a semantic analysis of the threat, cross-references local data, and outputs a strict JSON verdict regarding the threat level and recommended actions.
- **Tier 4: The Command Center (The Interface):** A Next.js front-end visualizes the telemetry in real-time. It features a dark-mode, glassmorphism UI with neon cyan and emerald green accents, displaying 3D node graphs of anomalous process trees and an interactive chat interface to interrogate the AI about specific security alerts.

5. Technical Stack

Component	Technologies Used	Purpose
Endpoint Agent	C/C++, eBPF, Go	Kernel-level event hooking and high-speed data transmission.
Backend & ML	Python, FastAPI, Scikit-Learn	Real-time event routing, baseline training, and anomaly scoring.
AI & RAG	Ollama, ChromaDB / FAISS	Local LLM inference and context retrieval.
Frontend UI	Next.js, React, Tailwind CSS	Real-time threat visualization and user interaction.
Data Storage	SQLite / TimescaleDB	Fast time-series storage for local telemetry.

6. Primary Use Cases

- **Zero-Day Threat Detection:** Identifying completely new malware based on what it *does* (e.g., rapidly modifying system files or opening reverse shells) rather than its file hash.
- **Living-off-the-Land (LotL) Mitigation:** Catching instances where legitimate system tools (like PowerShell, bash, or curl) are hijacked to perform unauthorized outbound network connections or payload executions.
- **Privacy-Preserving Security:** Allowing users to analyze advanced threats on their personal machines without sending proprietary source code, private documents, or system telemetry to cloud security vendors.

- **Automated Incident Response:** Using the structured JSON output from the local LLM to trigger automated kill-switches (e.g., terminating malicious PIDs or isolating network interfaces) instantaneously.

7. Future Expansion Scope

- **RAG-Powered Vulnerability Triage:** Integrating local NVD (National Vulnerability Database) feeds so the AI can map observed anomalous behavior directly to known CVEs and automatically generate Infrastructure-as-Code remediation playbooks.
- **Cross-Platform Parity:** Expanding the eBPF Linux agent to include Windows ETW (Event Tracing for Windows) support for a truly universal cross-platform endpoint agent.